

Weiss, ICCV 2001

Deriving Intrinsic Images from Image Sequences
algorithm reference

1. The model, and why it needs a prior

An image is the product of what the surface *is* and how the light *fell*:

$$I(x, y, t) = R(x, y) L(x, y, t)$$

with R the **reflectance** (constant over time) and L the **illumination** (changes every frame). At one pixel, T frames give T equations in $T + 1$ unknowns (R and $L_1, \dots, L_T$), so the problem is ill-posed no matter how many frames you collect — and $R \equiv 1$, $L_t = I_t$ always fits. Worse, for *any* field $\alpha(x, y)$ the pair $R' = R\alpha$, $L'_t = L_t/\alpha$ reproduces every frame exactly.

The prior. Derivative filter outputs of natural images are *sparse* — mostly zero, rarely huge — which is well modelled by a Laplacian $P(x) = \frac{1}{Z}e^{-\alpha|x|}$. Weiss assumes this of $f_n \star l$, the *filtered* illumination. He does **not** assume l itself is sparse; that weaker-looking change is the whole contribution of the paper (see §4).

2. Table 1 — the original (offline) algorithm

#	Algorithm	In words
0	given $I_1, \dots, I_T$; $f_1 = [-1 \ 1]$, $f_2 = [-1 \ 1]^\top$	T photos, static camera, only the lighting changes. Take two filters: horizontal and vertical derivative.
1	$i_t = \log I_t$	Log every frame. The product $I = RL$ becomes the sum $i = r + l$: a constant we want plus a nuisance that varies.
2	$o_n(x, y, t) = i_t \star f_n$	Filter every frame with both filters. <i>Stop looking at brightness, look at edges</i> — the prior is a statement about changes.
3	$\hat{r}_n(x, y) = \text{median}_t o_n(x, y, t)$	At each pixel, take the median down the time axis. A shadow edge crosses a given pixel in only a <i>minority</i> of frames, so it is an outlier and the median discards it; a paint edge is present in <i>every</i> frame and survives. (Claim 1.)
4	$\hat{r} = g \star \left(\sum_n f_n^r \star \hat{r}_n \right)$	We now hold the <i>edges</i> of the answer, not the answer. Integrate them back into an image by least squares (the two filters need not agree). With derivative filters this is a Poisson solve; g is precomputed once.

#	Algorithm	In words
5	$\hat{R} = \exp(\hat{r})$	Undo the log $\Rightarrow$ the reflectance image, shadows removed.
6	$\hat{L}_t = I_t / \hat{R}$	Free of charge: divide each original frame by the reflectance to get that frame's illumination image.

Steps 3 and 4 are equations (2) and (6) of the paper. The reversed filter is $f_n^r(x, y) = f_n(-x, -y)$ and g solves

$$g \star \left(\sum_n f_n^r \star f_n \right) = \delta.$$

For derivative filters $\sum_n f_n^r \star f_n$ is the negative discrete Laplacian, so step 4 is exactly $\nabla^2 \hat{r} = \text{div}(\hat{r}_1, \hat{r}_2)$. **g depends only on the chosen filters, never on the data**, so it is computed once and reused forever — which is exactly what makes Table 2 possible.

3. Table 2 — the online (streaming) algorithm

Identical to Table 1 except for step 3. Steps 1, 2, 4, 5, 6 are already per-frame and pointwise; the plain median is the *only* thing that needs the whole sequence, so it is the only thing that changes.

#	Algorithm	In words
0	<i>once, in advance:</i> allocate $H_n[x, y, b]$, $b = 0..255$; set $t = 0$; precompute g	One 256-bin histogram per pixel per filter, plus a frame counter. This is all the memory there will ever be — it does not grow. g never depends on the data, so it is built once.
<i>for each arriving frame I_t:</i>		
1	$i = \log I_t$	Same as before.
2	$o_n = i \star f_n$	Same as before.
3a	$H_n[x, y, \text{bin}(o_n)] += 1$; $t += 1$	The change. Instead of storing the value, drop it into a bucket and add one to that bucket's tally. $O(1)$ work, and the sample itself is thrown away.
3b	$\hat{r}_n = \text{smallest } b \text{ with } \sum_{b' \leq b} H_n[x, y, b'] \geq t/2$	The running median, read straight off the histogram: walk up the buckets adding counts, stop when you have passed half the frames. Sort order is free — it is the bin index.
4	$\hat{r} = g \star \left(\sum_n f_n^r \star \hat{r}_n \right)$	Same as before, using the <i>current</i> medians. Available at any frame.
5	$\hat{R}_t = \exp(\hat{r})$	Emit the reflectance image for this frame.
6	$\hat{L}_t = I_t / \hat{R}_t$; discard I_t	Emit the illumination, then throw the frame away. No frame is ever stored.

	Table 1 (offline)	Table 2 (online)
Memory	$O(t)$ frames — unbounded	$256 \times N$ counters per pixel — constant in t
Work/frame	re-sort every sample	$O(1)$ per pixel per filter
Output	only after the last frame	R_t every frame

4. Symbol legend

The five that cause the trouble

A filter is a tiny grid of numbers you slide over an image, multiplying and adding as you go, to produce a new image. It answers one fixed question at every pixel at once. Weiss’s two filters ask “how much did brightness change going right?” and “...going down?”

f_n — **the n -th filter itself**, i.e. those numbers. Not a logarithm of anything; there is no capital F in the paper, and the subscript indexes *which* filter, nothing more. Weiss uses $N = 2$ throughout:

$$f_1 = \begin{bmatrix} -1 & 1 \end{bmatrix} \text{ (horizontal),} \quad f_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \text{ (vertical).}$$

$\star$ — **convolution**, the act of sliding. With f_1 it literally reduces to “subtract each pixel from the one to its right”.

$o_n(x, y, t)$ — **the output** of filter n applied to the log frame t (o for *output*). One number per pixel, per filter, per frame: *how much the log image changed at that spot, in that direction, in that frame*. This is the data the algorithm actually works on — from step 2 onward, the brightness values are never touched again. With two filters and T frames you hold $2T$ such images.

The median — fix one pixel and one filter, then look *down the time axis* at that pixel’s T values of o_n and take the middle one. It is chosen because the Laplacian prior turns maximum-likelihood into minimising $\sum_t |o_n - c|$, and the minimiser of a sum of absolute deviations is the median. Its optimality condition counts how many samples sit on each side, never how far away they are — which is precisely why a handful of wild values (a shadow edge sweeping past in a few frames) cannot move it.

g — **the undo-filter**. After step 3 we hold the *edges* of the answer, $\hat{r}_n$, but we want the answer $\hat{r}$. g is the filter that reverses “take derivatives”: it satisfies $g \star (\sum_n f_n^r \star f_n) = \delta$, so applying it cancels the filtering out. For derivative filters it is the inverse Laplacian, i.e. a Poisson solve. Because it is defined purely by f_1 and f_2 , it is computed once, in advance, forever.

Everything else

Symbol	Meaning
I, R, L	the real images: input, reflectance, illumination
i, r, l	their logarithms. <i>Lowercase means log for these three letters only</i> — it is not a general rule, which is why f_n trips people up
r_n	shorthand for $r \star f_n$; the subscript is a <i>filter index</i> , not a log index
f_n^r	the reversed filter $f_n(-x, -y)$. The superscript r means <i>reversed</i> , not reflectance
$\hat{\cdot}$	a hat marks an <i>estimate</i> : r is the truth, $\hat{r}$ is our recovered version
δ	the identity filter: $\delta \star u = u$ for any u
N, T	number of filters ($N = 2$) and number of frames
$H_n[x, y, b]$	Table 2 only: how many frames so far put this pixel’s filter- n response into bin b

5. Why the median of the *filtered* frames, not of the frames

Take two pixels P and Q with *identical* true reflectance, where P is sunlit in every frame and Q is shaded in every frame.

Median of the images gives P a bright value and Q a dark one — two different answers for the same paint, and no number of extra frames repairs it, because every sample is biased the same way. This is the ML estimate only if l itself is sparse (“most pixels are unshadowed”), which is rarely true.

Median of the filtered images gives them the same answer, because a permanent illumination offset is *constant* and a constant has zero derivative — it never enters the filtered signal at all. This is the ML estimate if $f_n \star l$ is sparse (“the illumination is mostly smooth, with rare sharp edges”), which usually is true. The same failure kills the temporal **min** and **mean** filters: both need each pixel to be cleanly lit at some moment.

Claim 2 (robustness). The Laplacian need not hold exactly. Let $p_\epsilon = P(|f_n \star l| < \epsilon)$ measure sparseness directly. Since the median is within ϵ whenever more than half the samples are, the binomial formula gives

$$P(|\hat{r}_n - r_n| < \epsilon) \geq \sum_{k>T/2}^T \binom{T}{k} p_\epsilon^k (1 - p_\epsilon)^{T-k}.$$

The paper’s example (85% of filter outputs below 1% of maximum, i.e. $p_\epsilon = 0.85$) gives at least 0.93 from three frames and 0.97 from five. *Note:* the summation limits are transposed in the printed paper; the upper tail is what reproduces those numbers.

6. The online variant (exam Q3)

Question. Give an *online* algorithm for the reflectance image R_t from a stream $I_1, \dots, I_t$. Store at most 256 images (or equivalent). Hint: intensities lie in $[0, 255]$.

Where it blocks

Steps 1, 2, 4, 5 and 6 are all per-frame and pointwise, and g is precomputed. **Only step 3 is offline:** a textbook median must keep and sort every sample, so memory grows without bound. The whole question therefore reduces to *computing a running median in constant memory*.

The fix

The hint (“values in $[0, 255]$ ”, “store 256 images”) is naming a **256-bin histogram**: 256 slots per pixel, which yields a median with no samples retained, because sort order comes free from the bin index. The resulting procedure is **Table 2** above.

Answer. Per-pixel 256-bin histogram of the filtered log-frames $\rightarrow$ running temporal median $\rightarrow$ the fixed Weiss reconstruction. Memory $256 \times (\# \text{filters})$ counters per pixel, constant in t ; work $O(1)$ per pixel per frame; R_t emitted every frame.

Details worth a line each

- The median is taken on filter *responses*, not intensities. A difference of two values from a 256-level alphabet has ~ 511 possible values, so 256 bins costs at most half a bin of quantisation — negligible, and bounded.
- Two filters means two histograms. Counting bytes: a counter needs $\lceil \log_2 T \rceil$ bits against a pixel’s 8, so 256 bins per filter is 256 images’ worth under 8-bit accounting and two filters is $2 \times$ the stated budget; use 128 bins each, or invoke “or equivalent memory footprint”. Showing the arithmetic is the point.
- A 256-frame ring buffer is a legal alternative (and adapts to slow scene change) but costs full frames rather than counters, re-sorts per pixel per frame, and caps the evidence at 256 frames.
- $\hat{r}$ is recovered only up to an additive constant, hence $\hat{R}$ up to one global scale — fix it by convention (e.g. $\max \hat{R} = 1$). Clamp I away from 0 before the log.

7. Limits

- **Linear camera assumed.** $I = RL$ needs a linear response; the paper blames a webcam’s automatic gain control for its weaker outdoor results.

- **Independence assumed** over space and time, which is plainly false: shadow edges are long and connected, and consecutive frames are similar.
- **Static illumination is invisible.** A shadow that never moves across the sequence is mathematically indistinguishable from dark paint and stays baked into $\hat{R}$.
- **A sequence is required.** The single-image decomposition of Barrow and Tenenbaum remains open.